Object Tracking

Recurrent Neural Network (RNN)

VOT (Video Object Tracking)

- **On-line learning: FCNT**
- **No on-line learning: ROLO, GOTURN, SiamFC, SiamRPN, SiamRPN++, SiamFC++**

MOT (Multiple Object Tracking)

- **IoU Tracker, SORT, Deep SORT, ByteTrack, FairMOT, Tracktor, Siamese Track-RCNN**

VOS (Video Object Segmentation)

RNN (Recurrent Neural Network)

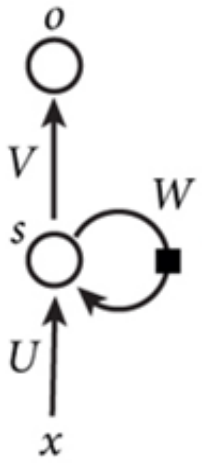
- **Idea**: to make use of sequential information (e.g., to predict a word in a sentence you better know which words came before it)
  - Recall: traditional neural networks assume all inputs/outputs are independent of each other

$$s_t = f_{w,u,v}\,(x_t,\ s_{t-1})$$

new state **old state**

Input vector

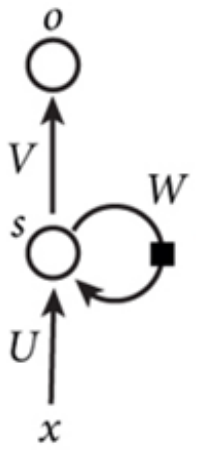
Some function with parameter w, u v



The same function and parameters

Process a sequence of vectors x by applying a **recurrence formula** at every time step.

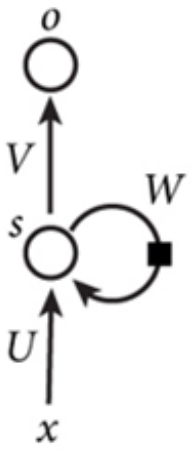
RNN (Recurrent Neural Network)

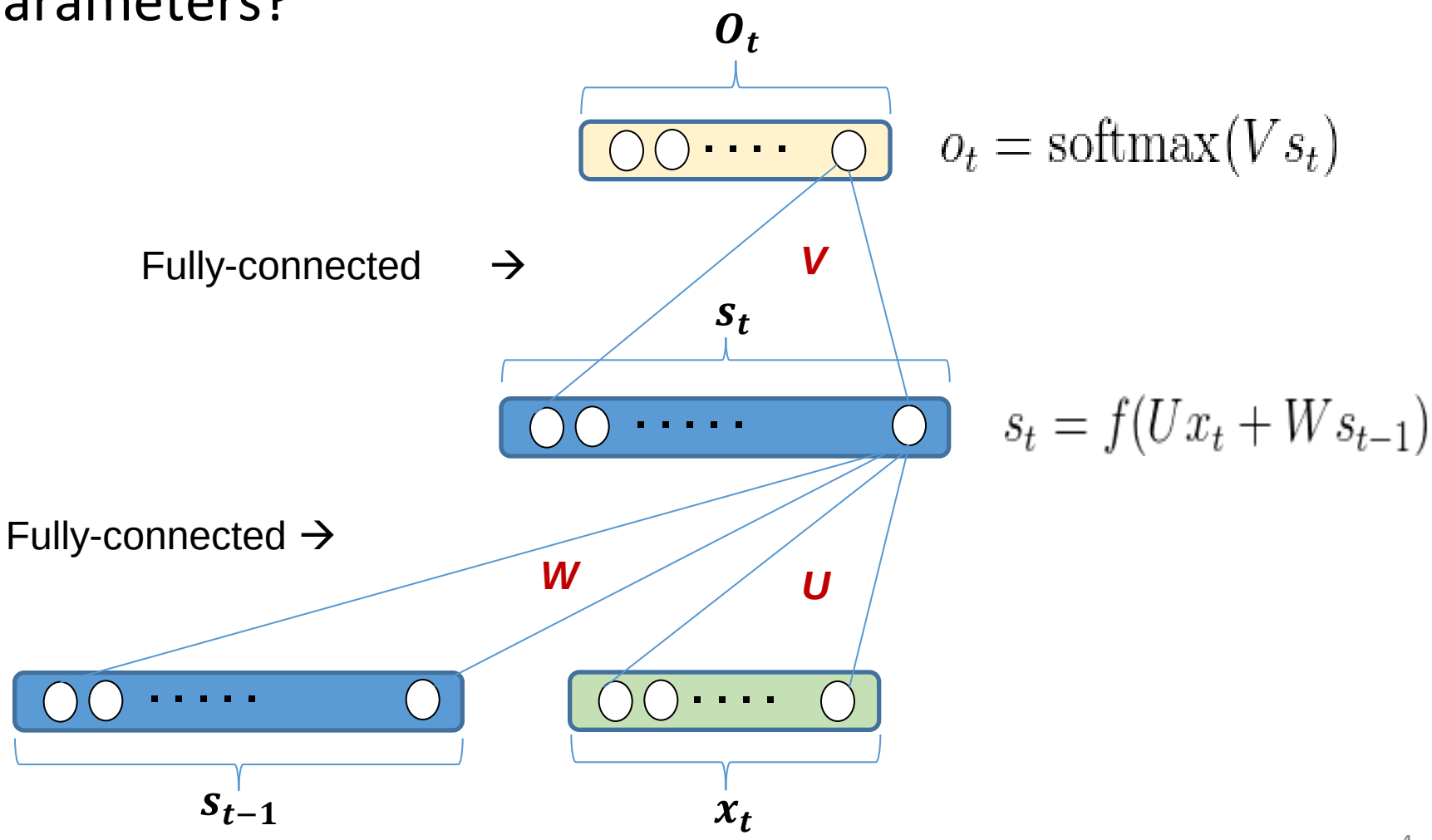


- The **input** (e.g., a vector (the word in a sentence))
  - The input at time t.
- The **hidden state** (e.g., $s_t = f(Ux_t + Ws_{t-1})$)
  - it is the "**memory**" of RNN. it captures information about what happened in all the previous time steps. (But in practice, it can't be too many time steps ago).
  - It is related to previous hidden state and the input at current step.
- The **output** (e.g., $o_t = \mathrm{softmax}(Vs_t)$)
  - it is calculated solely based on the hidden state at time t
  - E.g., to predict the next word in a sentence it will be a vector of probabilities across our vocabulary

RNN

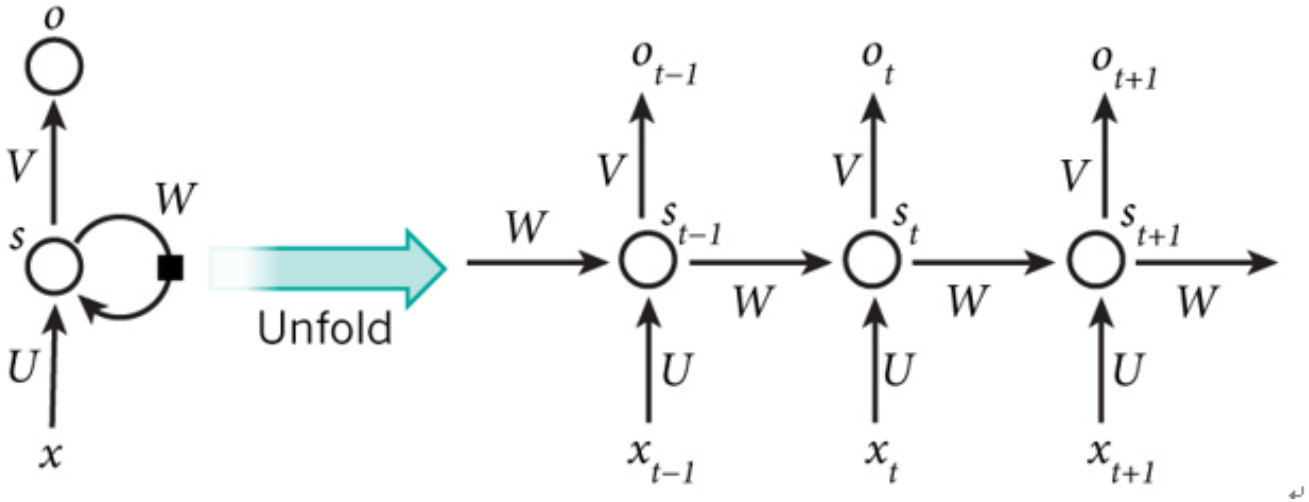
- Number of parameters?





RNN(Recurrent Neural Network)

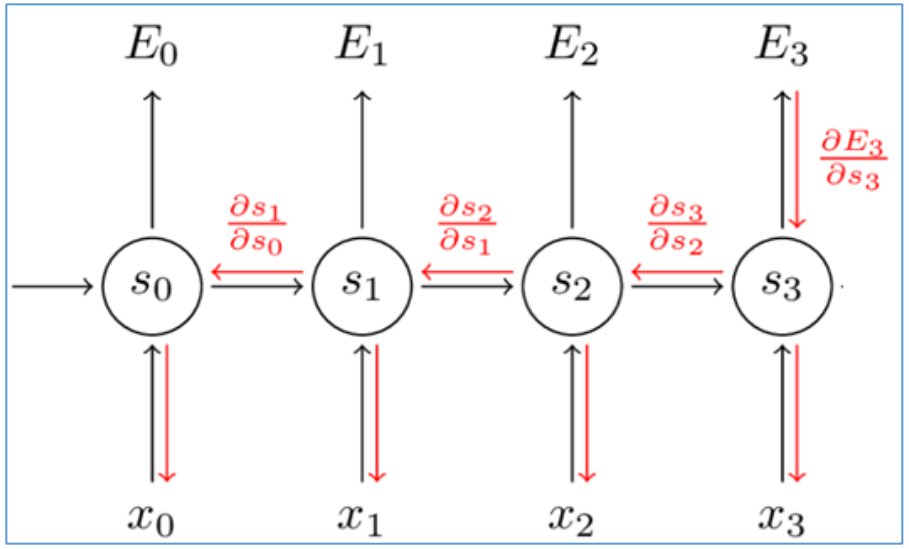
- **RNN unfolding** (unrolling)
  - E.g., To unroll the network for the complete sequence.
  - if the sequence we care about is a sentence of 5 words, the network would be unrolled into a 5 steps, one for each word.



A RNN shares the same parameters (U, V, W) across all steps.

RNN(Recurrent Neural Network)

- **Backpropagation Through Time (BPTT)**
  - It treats full sequence (sentence) as a single training example, so the total error is the sum of the errors at each time step for one example.
  - Use standard backpropagation but sum up gradient at each time step.
  - standard RNNs are hard to train:
    - Sequences can be quite long, (e.g., 20 words).
    - long-term dependency may cause vanishing gradient problem.
    - in practice many people *truncate* backpropagation to a few steps.



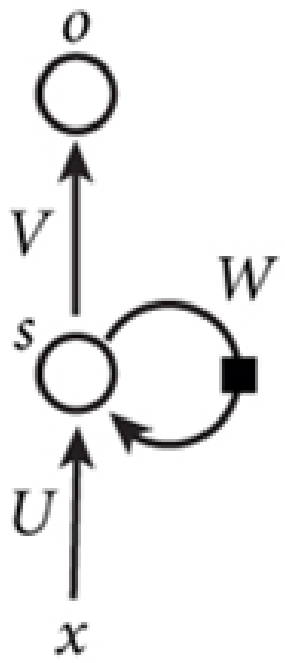
RNN – Vanilla example

- Vanilla RNN consists of a single “hidden” state

$$s_t = f_{w,u,v}\,(x_t,\, s_{t-1})$$

$$s_t = tanh(\, Ux_t + Ws_{t-1}\,)$$

$$o_t = Vs_t$$

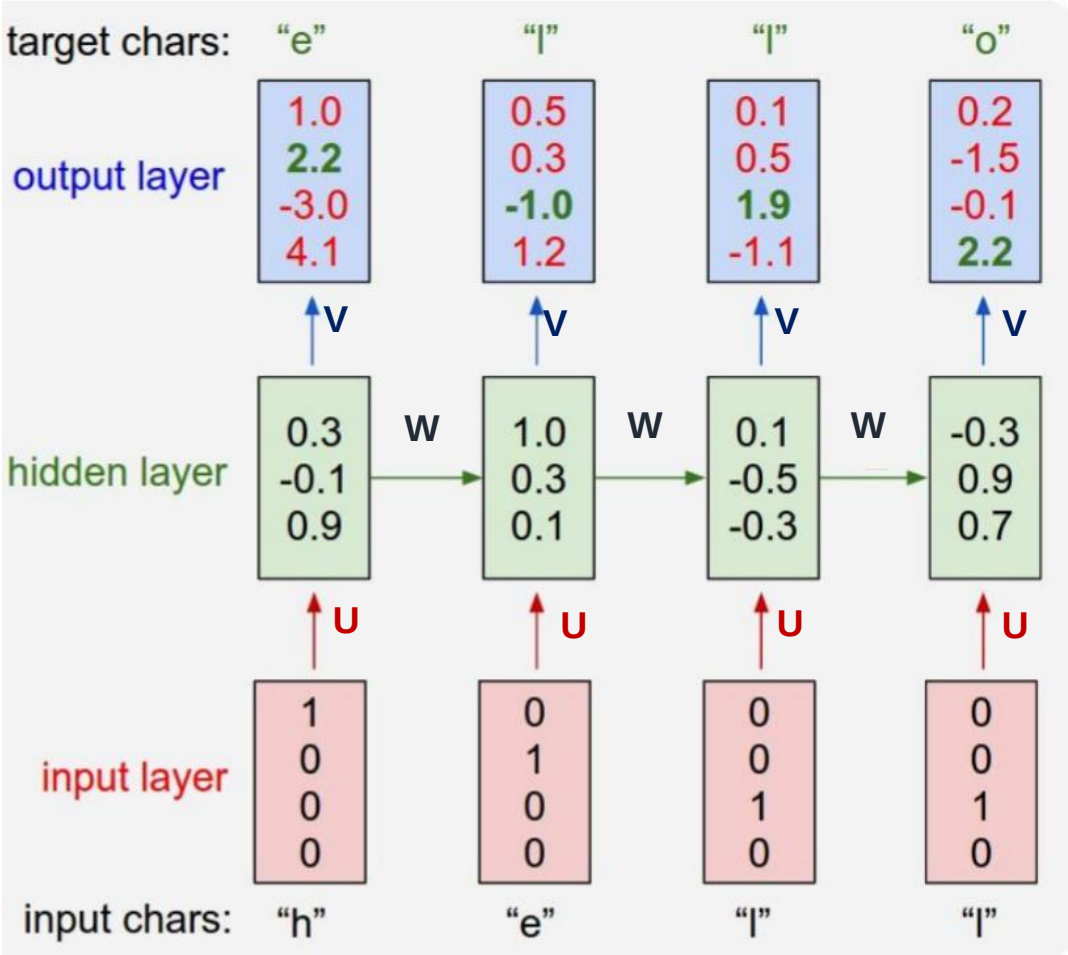


RNN – Vanilla example

- **Character-level language model:** predict next character
- Vocabulary: [h,e,l,o]
- training sequence:
  - e.g., “hello”

$$o_t = Vs_t$$

$$s_t = tanh(Ux_t + Ws_{t-1})$$



RNN: multiple layers

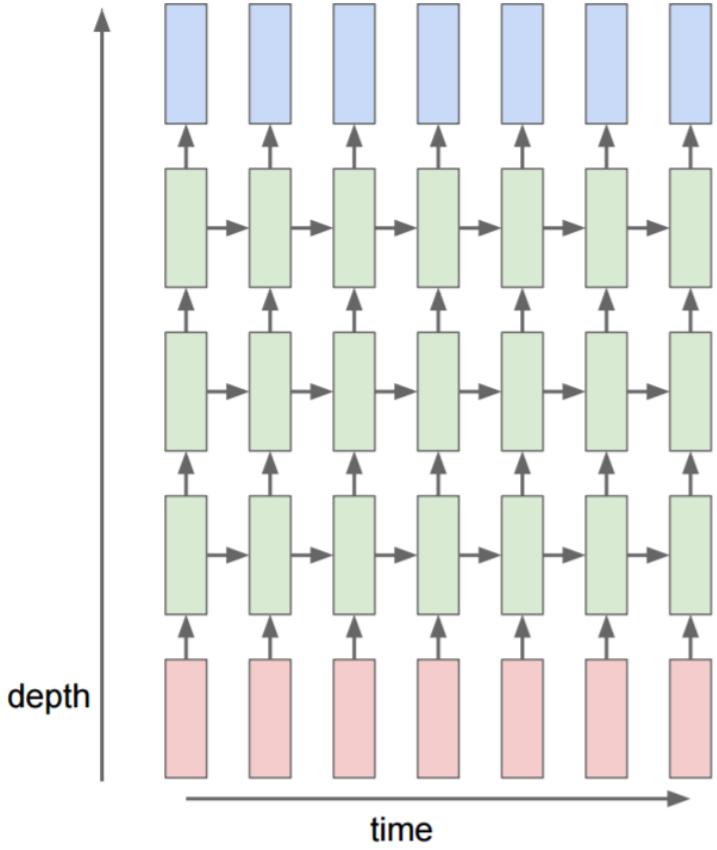
- RNNs with multiple layers: l = 1, ..., L.
- $\boldsymbol{h_t^l}$: the hidden state at time t on layer l.
  - is computed as

$$h_t^l = \tanh W^l \left(h_t^{l-1}, h_{t-1}^l\right)^\top$$

$h \in \mathbb{R}^n$.

W^l $[n \times 2n]$

  - W is a matrix of learnable parameters, which varies on each layer of the network.
  - $\boldsymbol{h^0}$ can be thought of as the input layer, holding the input vector,
  - $\boldsymbol{h^L}$ holds the final embedded representation used to produce output.



RNN(Recurrent Neural Network)

- ***Vanishing Gradient problem***
  - Gradient values are shrinking fast through propagation, eventually vanishing completely after a few time steps.
- However, relevant information maybe apart from far away
  - Ex1: The clouds are in the ***sky***
    - gap between relevant information is small, easy to predict the next: “sky”
  - Ex2: I grew up in France… I speak fluent ***French***
    - Recent information suggests that next word is the name of a language, but if we want to predict which language, we need to go further back.

➔ LSTM (Long Short Term Memory)

LSTM (Long Short Term Memory)

- **f (forget gate)**: how much of ***previous cell state*** we are going to throw away (all 1's: completely keep it; all 0's: completely drop it.)
- **i (input gate)**: how much of ***newly computed cell state*** to be stored.
- **o (output gate)**: how much of internal state you want to ***output*** (higher layers or next time step)

$$f = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$ forget gate

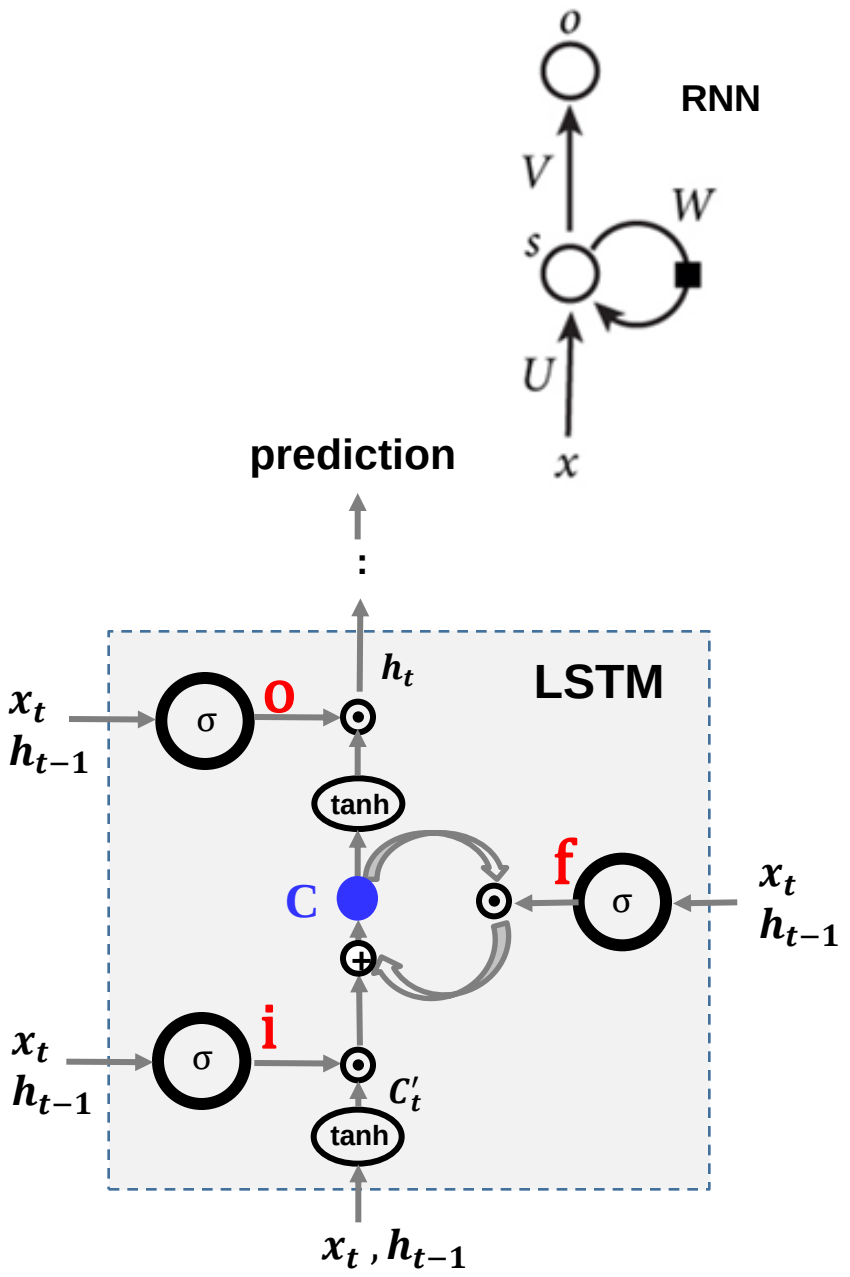
$$i = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$ input gate

$$o = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$ output gate

$$C'_t = tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$ candidate cell state

$$C_t = C_{t-1} \circ f + C'_t \circ i$$ Updated cell state

$$h_t = tanh(C_t) \circ o$$ hidden gate



LSTM

$$f = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$ forget gate

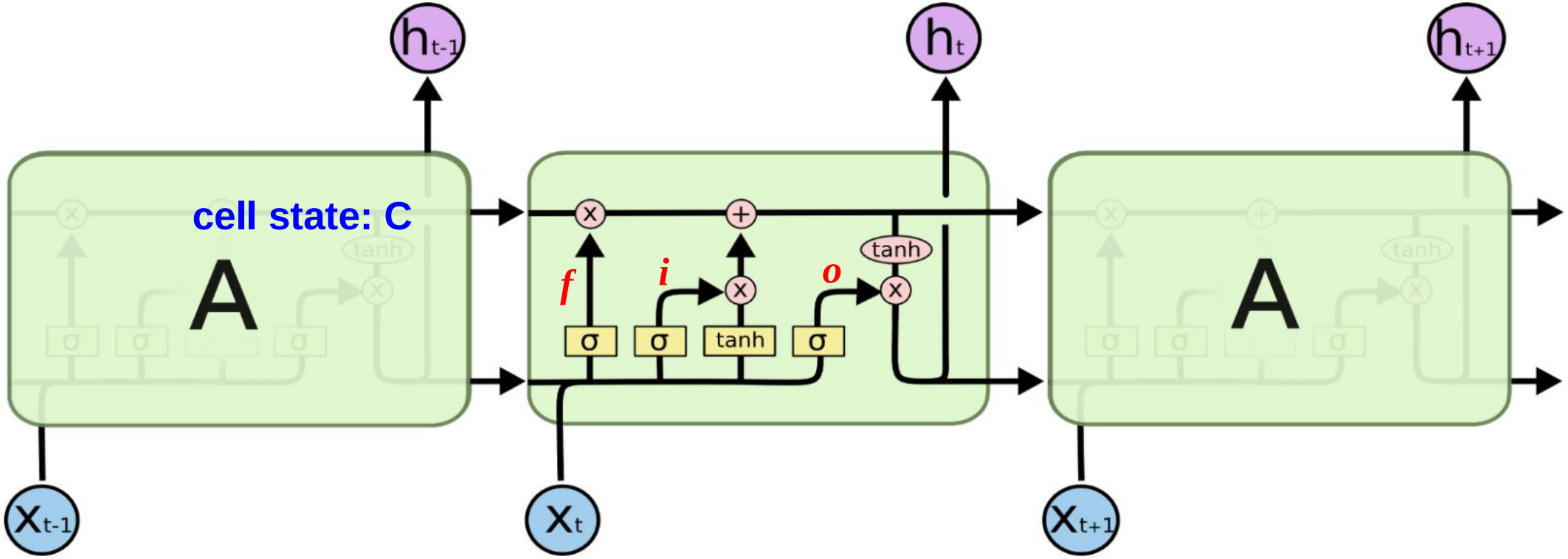
$$i = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$ input gate

$$o = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$ output gate

$$C'_t = tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$ candidate cell state

$$C_t = C_{t-1} \circ f + C'_t \circ i$$ Updated cell state

$$h_t = tanh(C_t) \circ o$$ hidden gate



LSTM Variations

- **LSTM with "peephole"**
  - Introduced by Gers&Schmidhuber[2000]
  - adding "peephole" connections. ➔ let the gates look at cell state
  - Instead of adding peepholes to all the gates (i, f, o), many papers give some gates peepholes and not others.

$$f = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$
$$i = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$o = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$C'_t = tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$
$$C_t = C_{t-1} \circ f + C'_t \circ i$$
$$h_t = tanh(C_t) \circ o$$

standard LSTM

$$f = \sigma(W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$
$$i = \sigma(W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$
$$o = \sigma(W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$$
$$C'_t = tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$
$$C_t = C_{t-1} \circ f + C'_t \circ i$$
$$h_t = tanh(C_t) \circ o$$

add peephole

LSTM Variations

- **Coupled forget and input gates**
  - Make decision of how much to forget and how much to add together
    - We only forget when we're going to input something in its place.
    - We only input new values to the state when we forget something older.

$$f = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$
$$i = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$o = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$C'_t = tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$
$$C_t = C_{t-1} \circ f + C'_t \circ i$$
$$h_t = tanh(C_t) \circ o$$

standard LSTM

$$f = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$
$$\cancel{i = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)}$$
$$o = \sigma(W_o \cdot [, h_{t-1}, x_t] + b_o)$$
$$C'_t = tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$
$$C_t = C_{t-1} \circ f + C'_t \circ (1 - f)$$
$$h_t = tanh(C_t) \circ o$$

coupled forget & input

LSTM Variations

- **GRU** (Gated Recurrent Unit), by Cho. et al. [2014]
  - coupled forget and input gates ➔ update gate
  - merges cell state and hidden state
  - If set the reset gate to all 1's and update gate to all 0's → plain RNN
  - simpler than standard LSTM ➔ growing popular.

$$f = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$
$$i = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$o = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$C'_t = tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$
$$C_t = C_{t-1} \circ f + C'_t \circ i$$
$$h_t = tanh(C_t) \circ o$$

standard LSTM

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$ **reset gate**

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$ **update gate**

$$h'_t = tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) \circ h_{t-1} + z_t \circ h'_t$$

GRU

Object Tracking

Recurrent Neural Network (RNN)

VOT (Video Object Tracking)

- **ROLO, GOTURN, SiamFC, SiamRPN, SiamRPN++, SiamFC++**

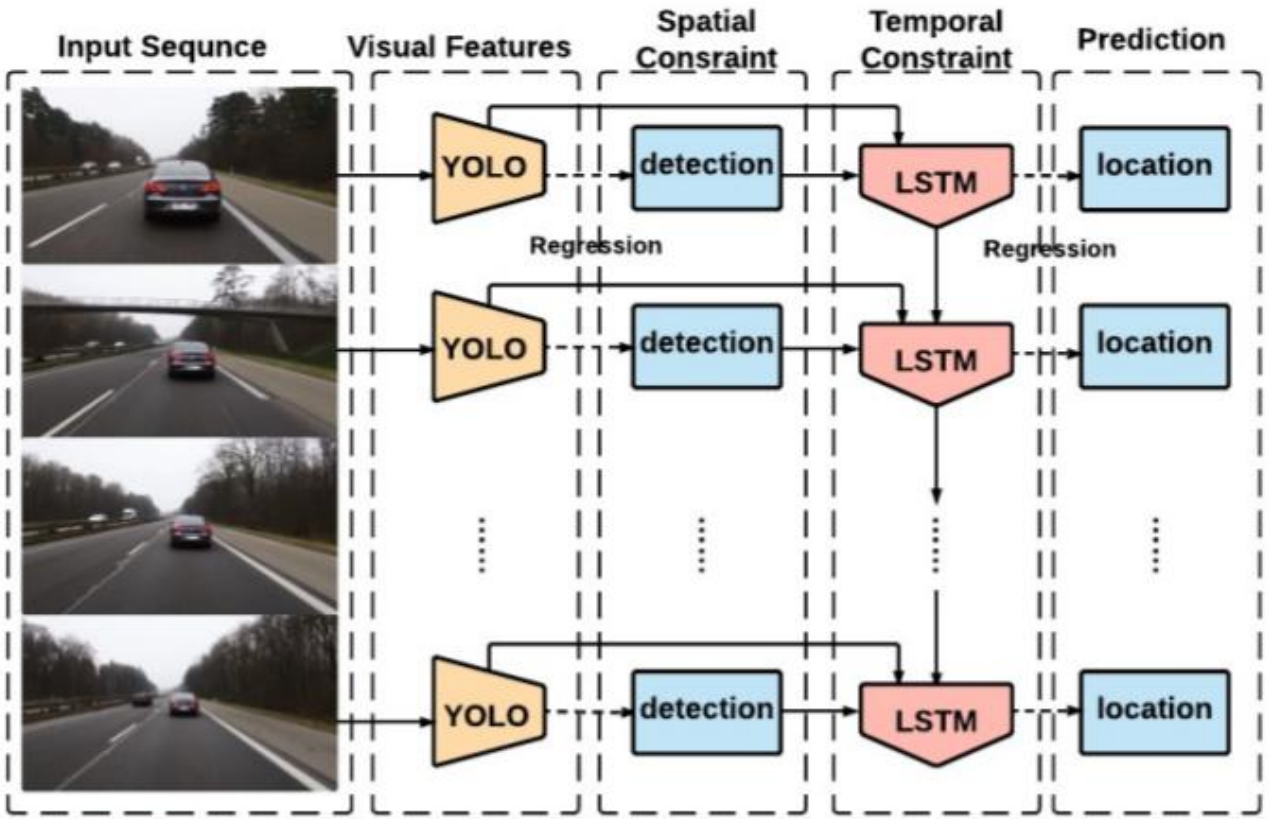
No on-line learning

MOT (Multiple Object Tracking)

- **IoU Tracker, SORT, Deep SORT, ByteTrack, JDE, FairMOT, Tracktor, Siamese Track-RCNN,**

VOS (Video Object Segmentation)

ROLO : Spatially Supervised Recurrent Convolutional Neural Networks for Visual Object Tracking



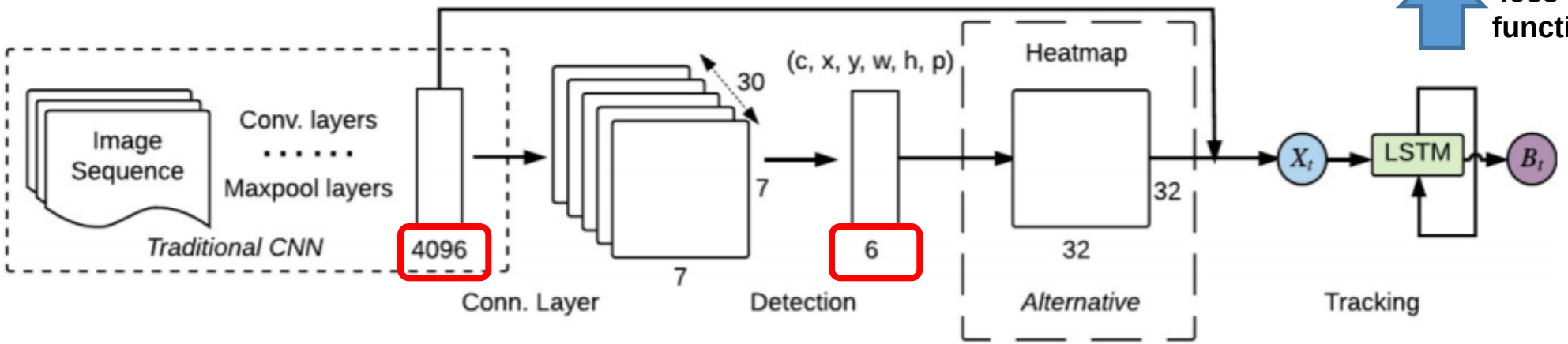
- (x, y, w, h, c, p) ∈ [0, 1]
- they are concatenated with the 4096-dimensional visual features, which will be fed into the tracking module LSTM .

ROLO (Recurrent YOLO)

(x, y, w, h)

$$L_{MSE} = \frac{1}{n}\sum_{i=1}^{n} ||B_{target} - B_{pred}||_2^2$$

loss function



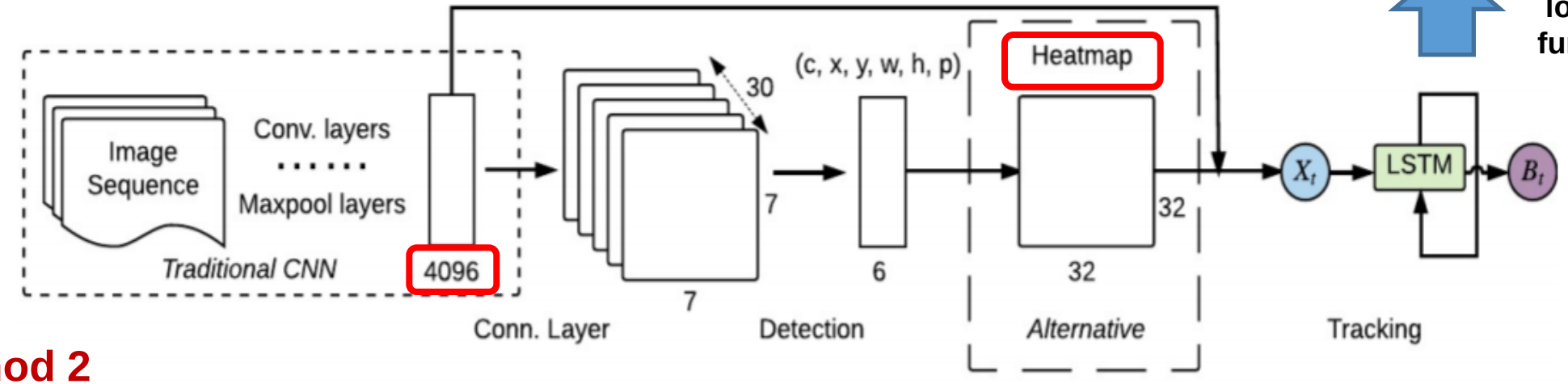
Method 1

- YOLO may detect multiple objects, the one with max IoU with target object is selected
  - 1st frame: use ground-truth object as the target.
  - Other frames: object being tracked on previous frames as the target.
- For the selected (single) object, concatenate its **prediction location** and **visual features (4096-d)** before feeding to LSTM.
- LSTM predicts new locations for the selected (tracked) object.

ROLO (Recurrent YOLO)

$$L_{MSE} = \frac{1}{n}\sum_{i=1}^{n} ||H_{target} - H_{pred}||_2^2$$

loss function



Method 2

- Translate the **prediction location** into a **32x32 heatmap**
  - Assign 1 to the bbox region while 0 elsewhere, and then warp it to a 32x32 map
- Convert the map into a 1024d vector, concatenate it with the **4096d visual features**, and then feed it to the LSTM.
- LSTM predicts a new heatmap (1024d vector) of the tracked object.
  - Loss function: MSE between predicted and target heatmaps

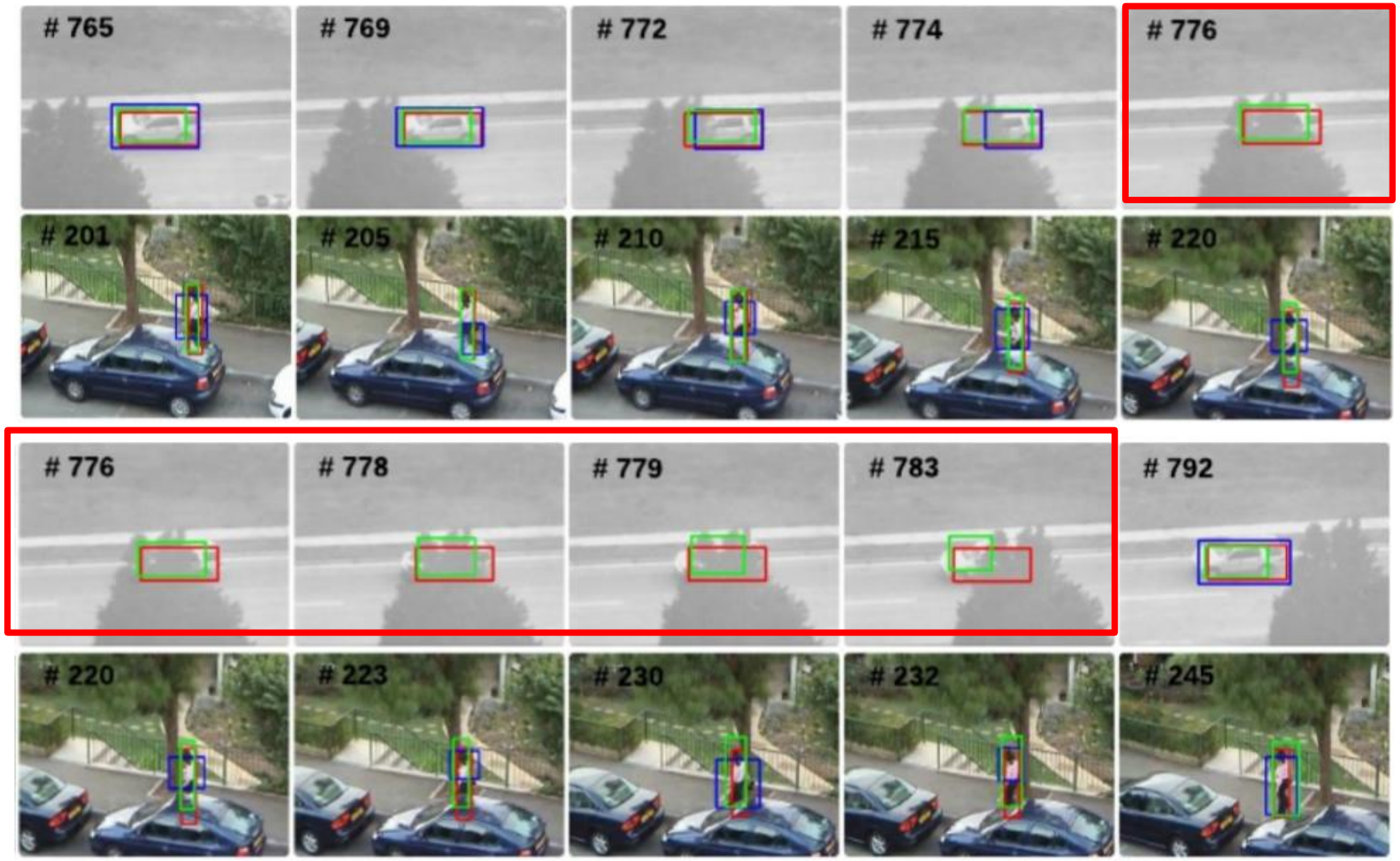
ROLO

20fps/60fps for YOLO/LSTM

8 cores of 3.4GHz Intel Core i7-3770 and an NVIDIA TITAN X GPU

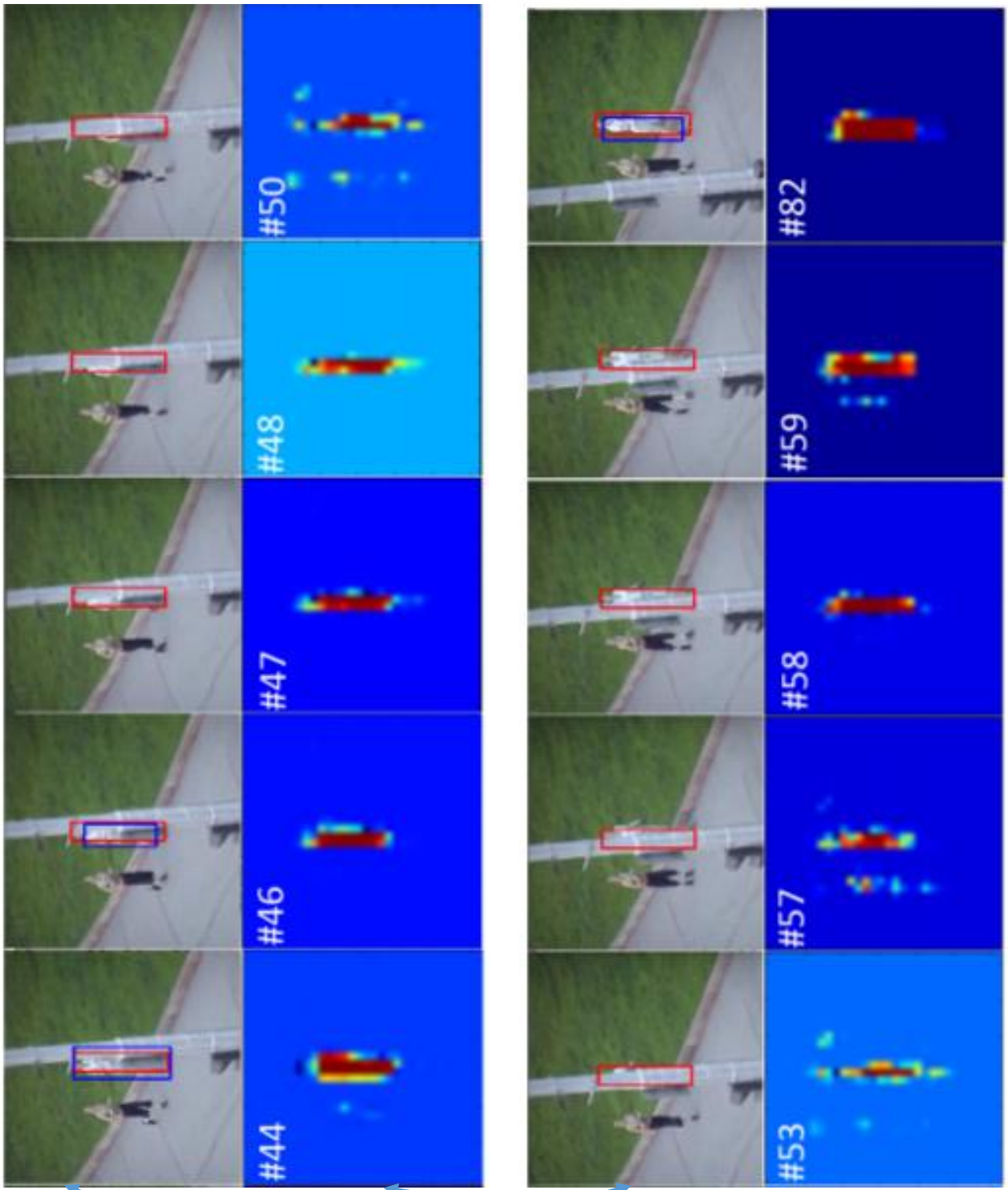
Red: GT,
Blue: YOLO
Green: ROLO

ROLO - Result



Red: GT,
Blue: YOLO
Green: ROLO

Occlusion Handling

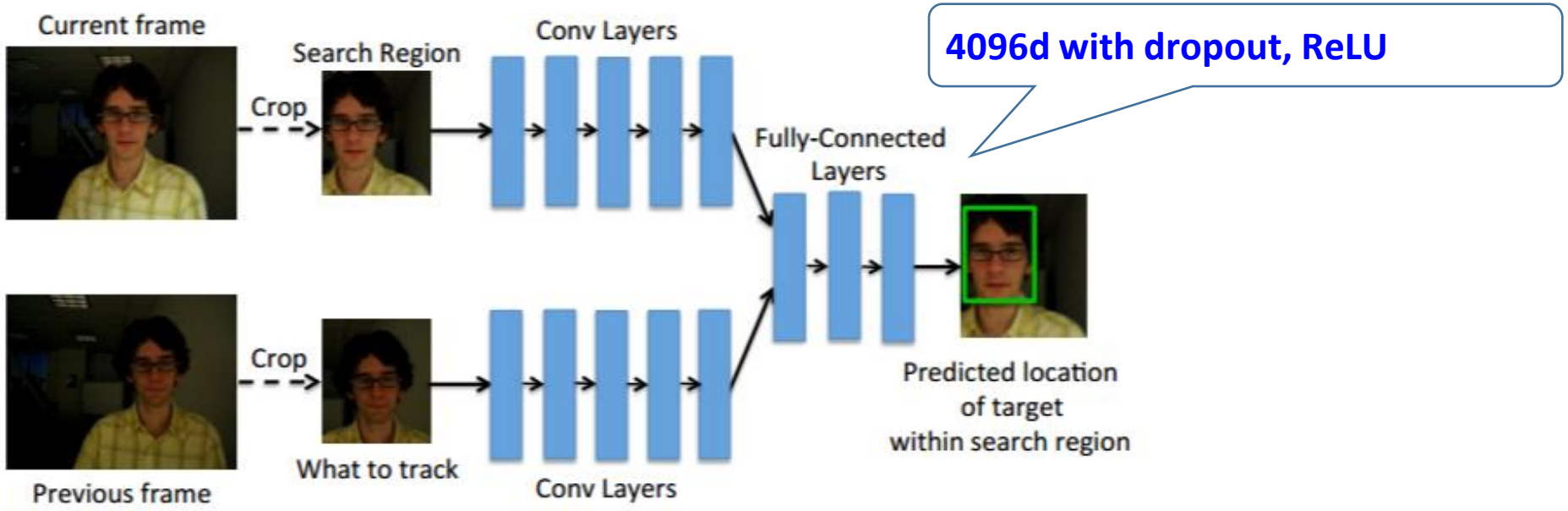


Red: GT,
Blue: YOLO

heatmap
predicted by
ROLO

GOTURN: Learning to Track at 100 FPS with Deep Regression Networks

- Feed the network a search region from the current frame and a target from the previous frame. The network learns to compare these crops to find the target object in the current image



GOTURN

- **What to track** (tracking target)
  - At frame t − 1, the tracker predicted the target centered at C = (c_x, c_y) with bbox size (w, h).
  - At time t, we take a crop of frame t−1 centered at *C* with size $k_1 w$ x $k_1 h$
    - k_1 is for preserving some context surrounding target
  - Scale the crop to meet the input size of CNN
- **Where to look** (search region)

$k_1 = k_2 = 2$

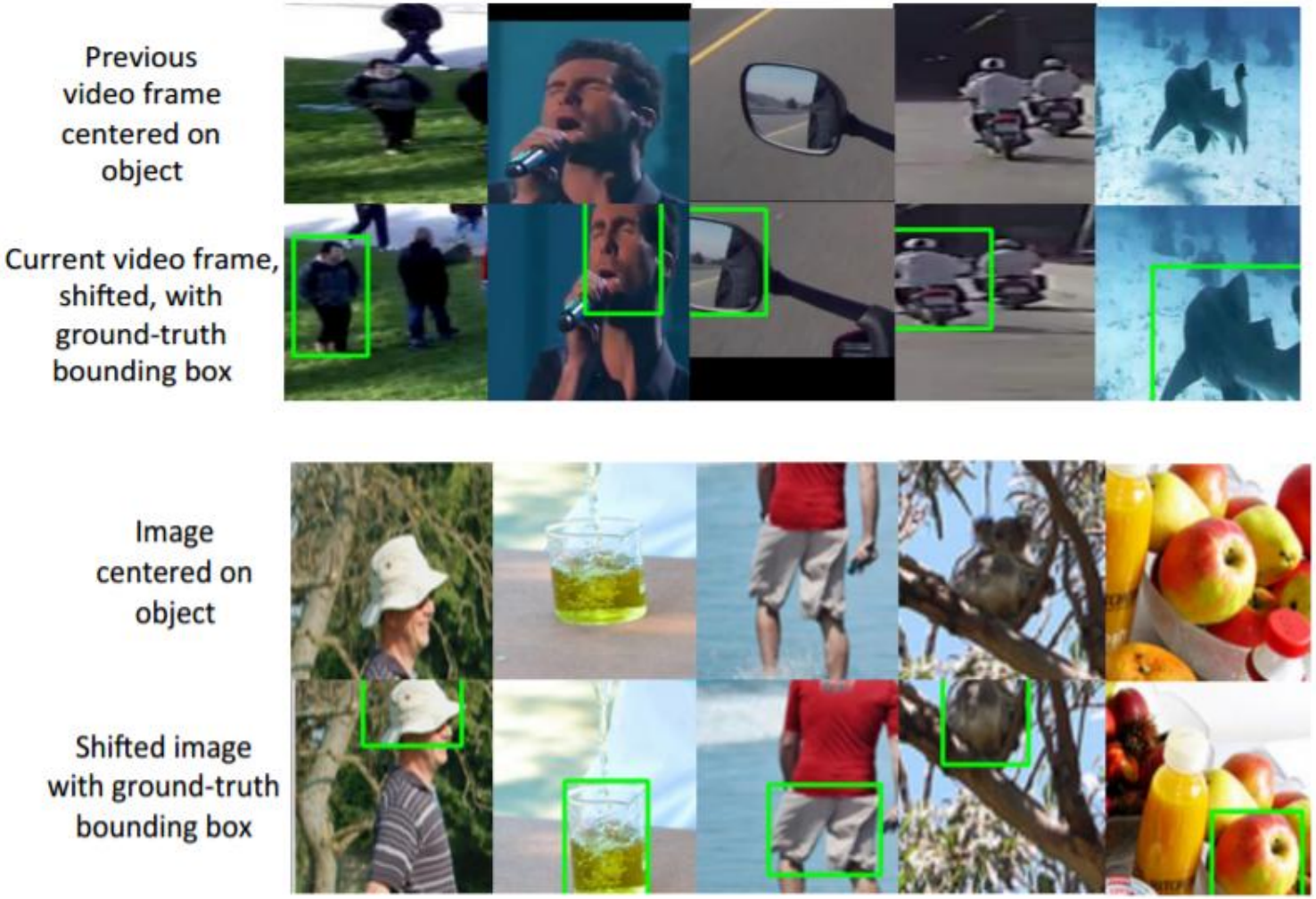
  - Search region is centered at C of current frame, with a size of $k_2 w$ x $k_2 h$.
    - k_2 is the search radius
- **Output**
  - top-left and bottom-right coordinates of the target in the search region

GOTURN

- Training

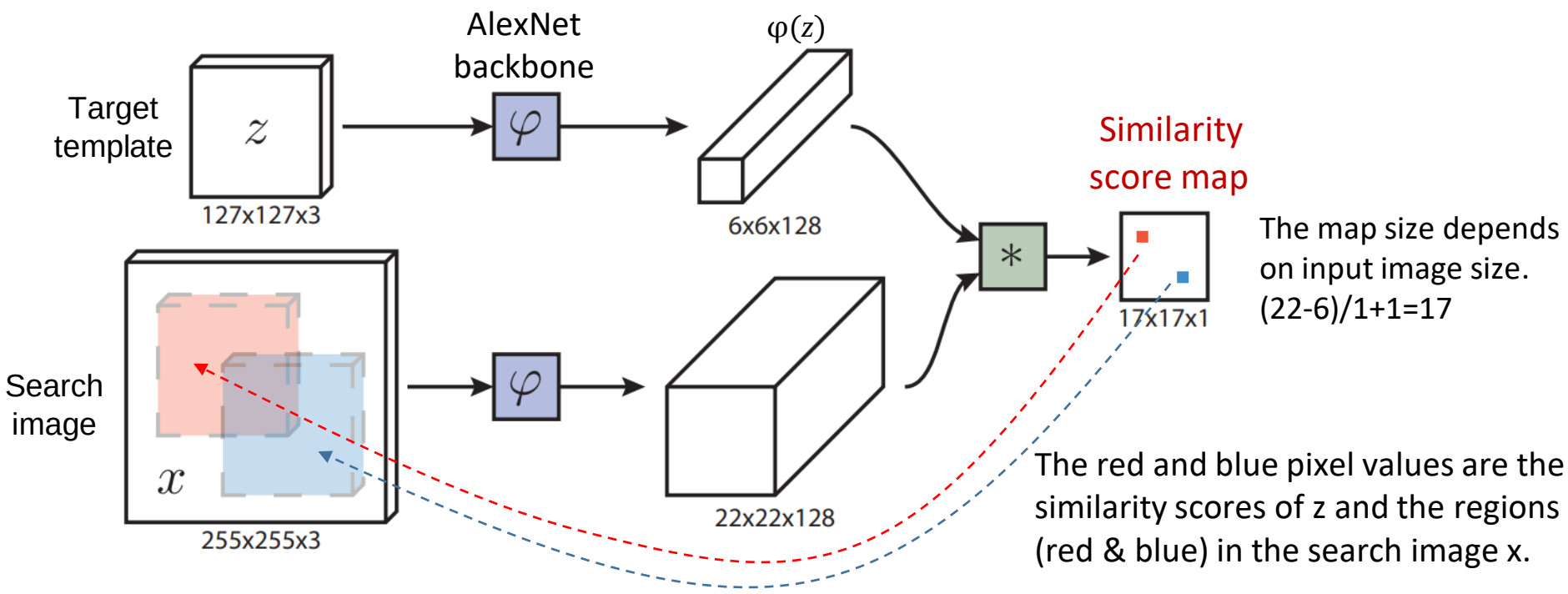
Use both video and image dataset

Randomly crop the images in training to create a translation or scale change



SiamFC: Fully-Convolutional Siamese Networks for Object Tracking

- Architecture: fully convolutional network
  - **Siamese network** φ : two branches share weights (AlexNet w/o padding & fc-layers)
  - **Cross-correlation** $*$: use $\varphi(z)$ as kernel to conv on $\varphi(x)$
    - **$\varphi(z)$ changes with input z** (Note. standard conv kernels are fixed after training.)



SiamFC

- Search range
  - The search region is centered at the previous position of target, with a region of approximately four times its previous size.
- Tracked object position
  - Upsample score map (17×17 → 272×272) to have more accurate localization
  - The position corresponds to the maximum score in the map is the tracked object center
- Scale variation handling
  - Search for the object over five scales $1.025^{\{-2, -1, 0, 1, 2\}}$
  - SiamFC-3s searches over 3 scales instead of 5 (86fps for 3 scales, 58fps for 5)

SiamFC

$$y[u] = \begin{cases} +1 & \text{if } k\|u - c\| \leq R \\ -1 & \text{otherwise} \end{cases} .$$

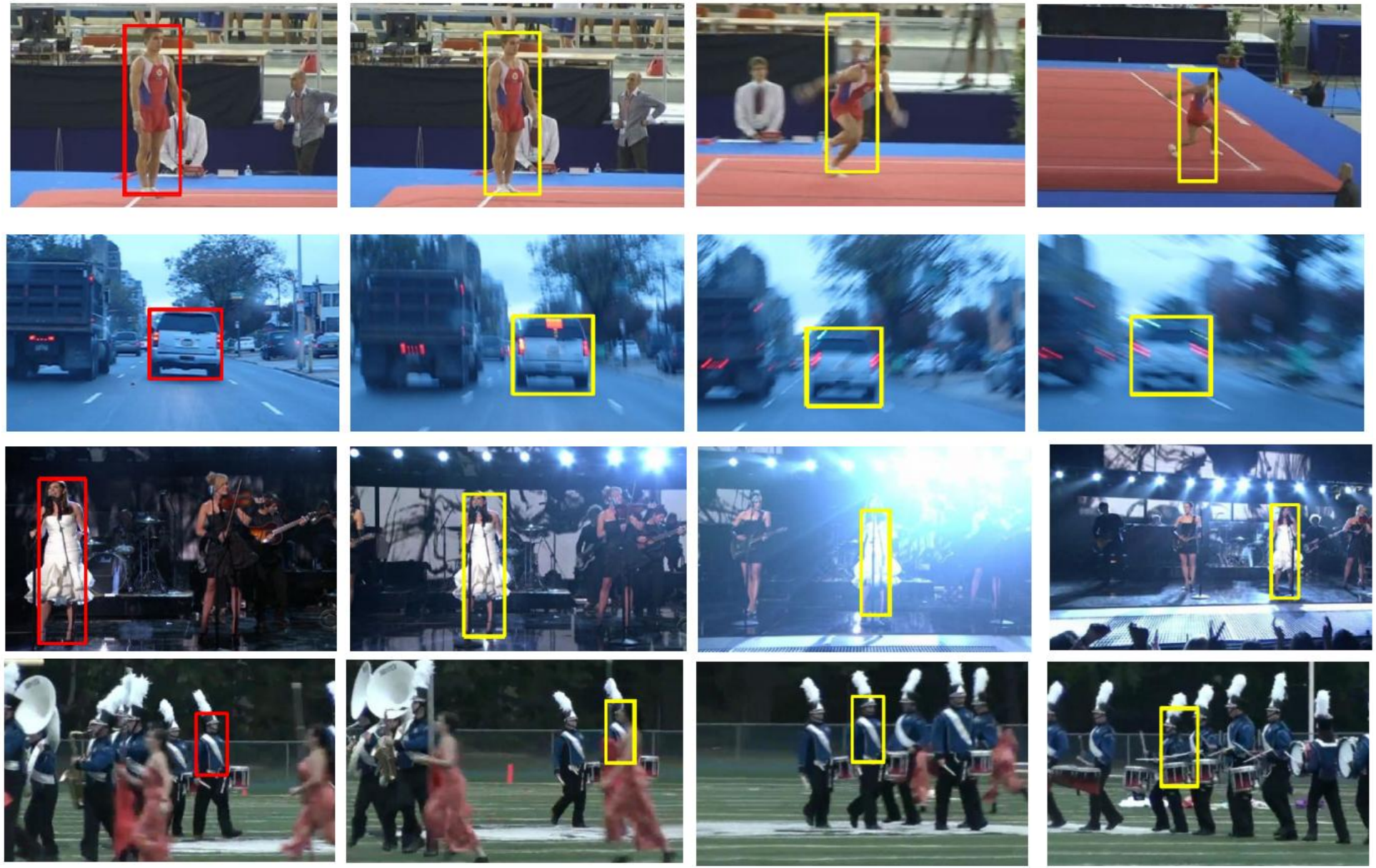
- Training
  - The elements of the score map are considered to belong to a positive example if they are within radius R of target center (k: stride of the network)
  - losses of the positive and negative examples in the score map are weighted to eliminate class imbalance
- Tracking
  - No on-line fine-tuning (model updates occur only during training)
    - only equipped with offline-learnt generalization capability
  - Always use target in the 1st frame as tracking target
    - initial object appearance is computed only once ($\varphi(z)$), and is compared to sub-windows of all the subsequent frames.
    - Updating of target does not gain much performance and thus keep it fixed.

Appearance change (good)

Motion blur (good)

Illumination change (good)

Occlusion (poor)



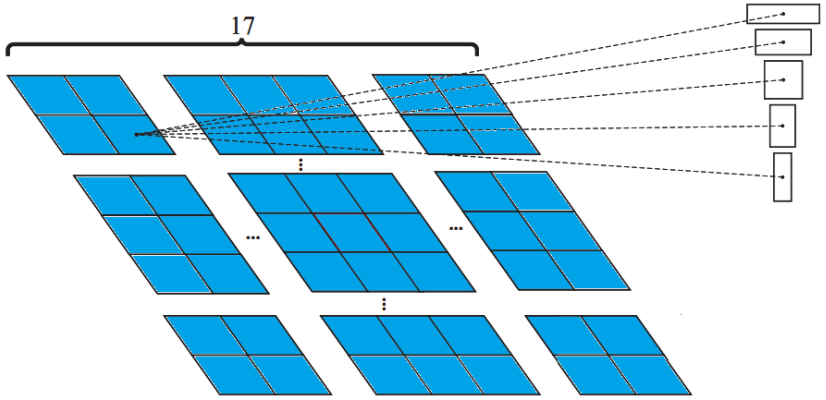
Frame 1 (init.) Frame 50 Frame 100 Frame 200

SiamFC

- Compared to GOTURN
  - SiamFC uses densely supervised heatmap
  - GOTURN uses one proposal regression
- Fully-convolutional : input image can be any size
- Based on similarity learning
- 86 fps on VOT-15

SiamRPN: High Performance Visual Tracking with Siamese Region Proposal Network

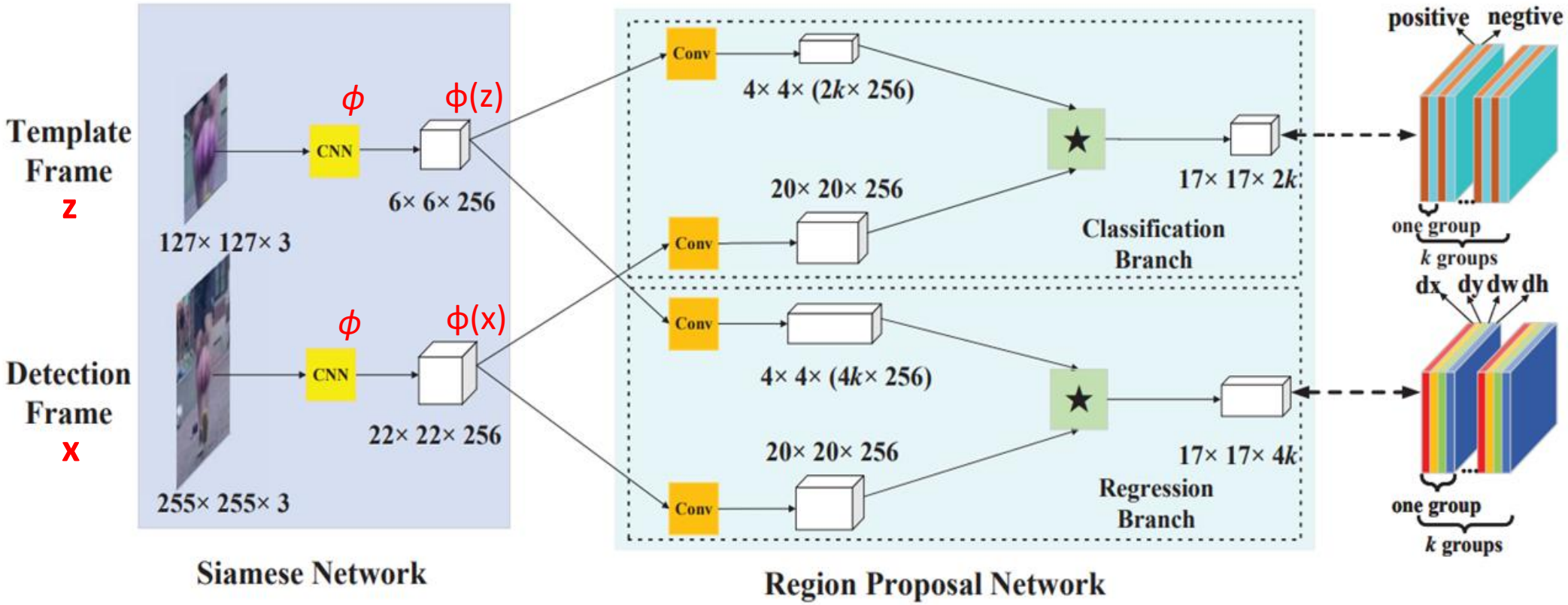
- SiamRPN: using bbox regression makes the tracking localization more precisely.
  - Siamese-FC is lack of bbox regression and needs to do multi-scale test which makes it less elegant.
- Trained on even larger video dataset
  - **Youtube-BB** is a large-scale video dataset consists of 380,000 15-20s video clips with densely-sampled high-quality single-object Bbox annotations.



SiamRPN

Φ : two branches share weights

- SiamRPN = Siamese network + Region Proposal network (RPN)
  - K anchors → K bbox prediction



SiamRPN

- Siamses network: feature extraction
  - Template branch: z: target $z \rightarrow \phi(z)$
  - Detection branch: x: current frame $x \rightarrow \phi(x)$

  $\phi(z)$ and $\phi(x)$ share parameters
- RPN network (assume **k anchors**)
  - Classification branch:
    - Increase #(channels) of $\phi(z)$ to 256 x 2k
    - keep same #(channels) of $\phi(x)$ as 256
  - Bbox regression
    - Increase #(channels) of $\phi(z)$ to 256 x 4k
    - keep same #(channels) of $\phi(x)$ as 256

Correlation

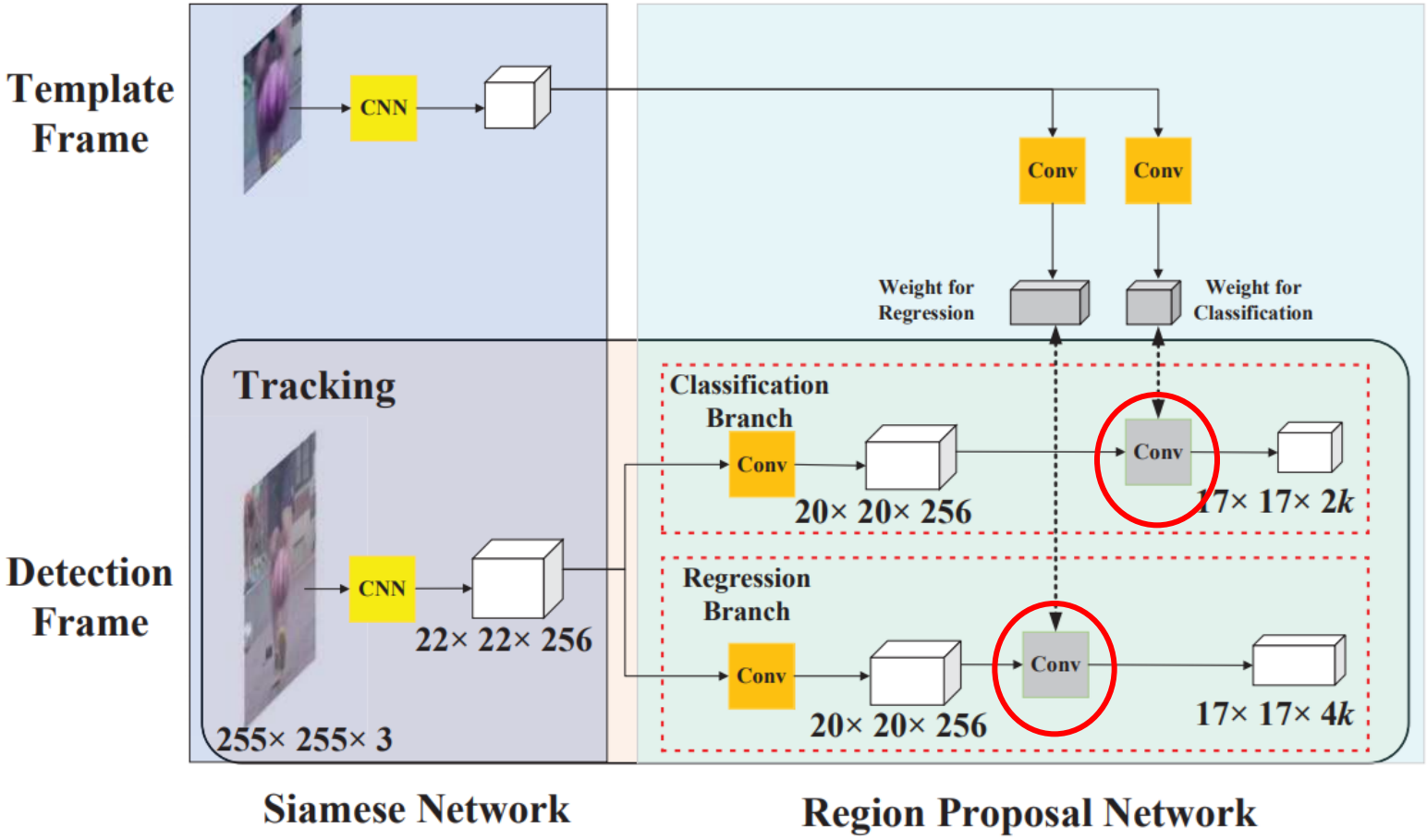
Conv. → 2K maps: fg score, bg score — x k anchors

Conv. → 4K maps: x, y, h, w — x k anchors

- During Training
  - Anchor: One scale with 5 aspect ratio: 0.33, 0.5, 1, 2, 3 (k=5)
  - Anchor is foreground: IoU > 0.6, anchor is background: IoU < 0.3

SiamRPN

- Tracking as one-shot detection



Template branch learns kernel weights for RPN using the first frame. Then the template branch is pruned.→ So the framework turns into a detection network.

SiamRPN

Tracking target center in the previous frame

- Proposal selection strategies
  - Discard the proposals too far away from the center (spatial penalty)
  - Re-rank the proposals' score by using scale change penalty (temporal penalty)

$$penalty = e^{k*max(\frac{r}{r'}, \frac{r'}{r})*max(\frac{s}{s'}, \frac{s'}{s})}$$

r : proposal's ratio of h and w
r' : r of the last frame
s : size of the proposal
s' : s of the last frame
k : hyper-parameter

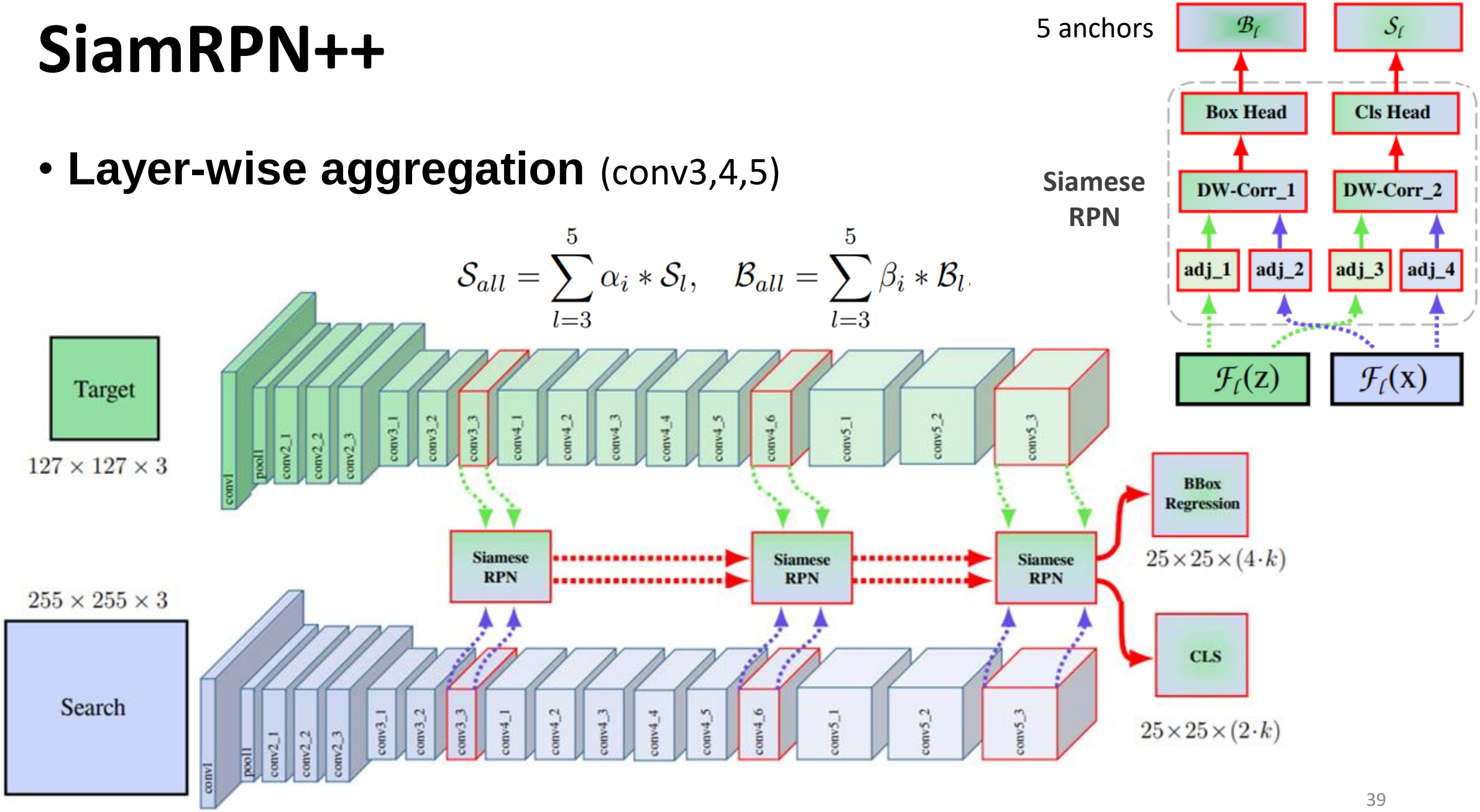
SiamRPN++: Evolution of Siamese Visual Tracking with Very Deep Networks

- ResNet-driven Siamese Tracking
  - ResNet-50 as its backbone (sharable)
  - Increase feature map: Reduce the network stride of ResNet from 32 to 8 and keep conv3,4,5 the same spatial sizes (*i.e.*, $\frac{H}{8} \times \frac{W}{8}$)
  - 1×1 conv (non-sharable) is added to conv3,4,5 to reduce #(ch) to 256
- Layer-wise aggregation
  - using multi-level features: from conv3,4,5
  - conv3,4,5 features are fed into three Siamese RPN module individually
  - Three RPN outputs are combined by a weighted-fusion layer
- Depth-wise cross correlation

SiamRPN++

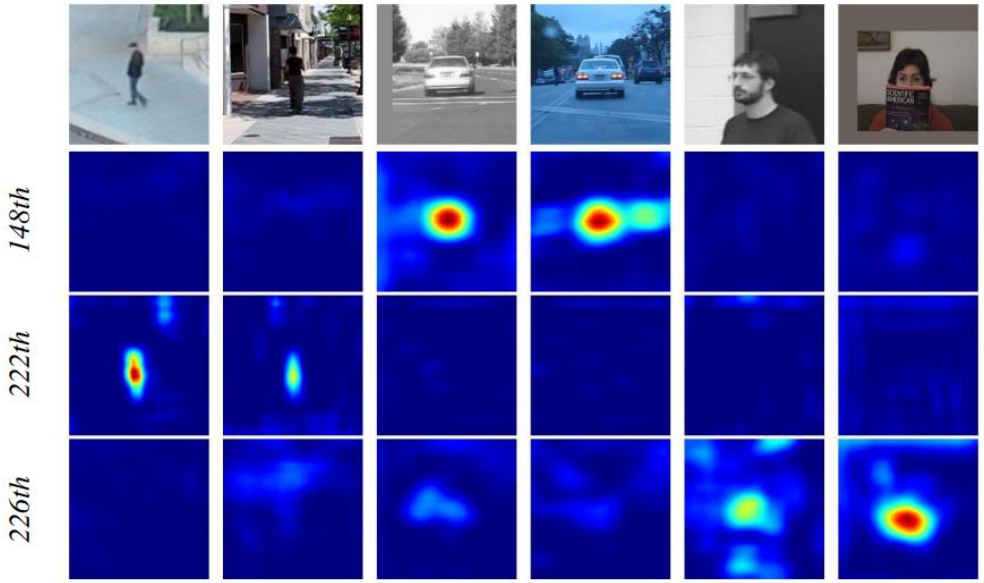
- **Layer-wise aggregation** (conv3,4,5)

$$\mathcal{S}_{all} = \sum_{l=3}^{5} \alpha_i * \mathcal{S}_l, \quad \mathcal{B}_{all} = \sum_{l=3}^{5} \beta_i * \mathcal{B}_l.$$



SiamRPN++

- **Depthwise Cross Correlation**
  - much fewer parameters than UP-XCorr

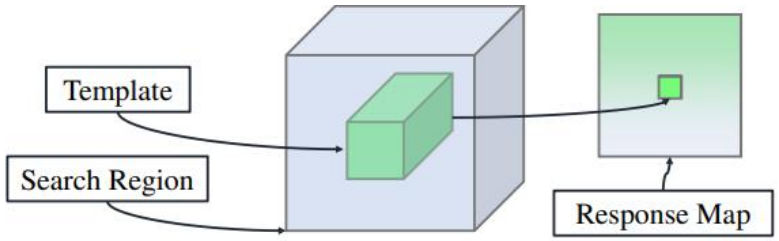


Depthwise correlation output in conv4

148th channel has high response on cars
222th channel has high response on persons
226th channel has high response on faces

SiamFC

single-channel response map

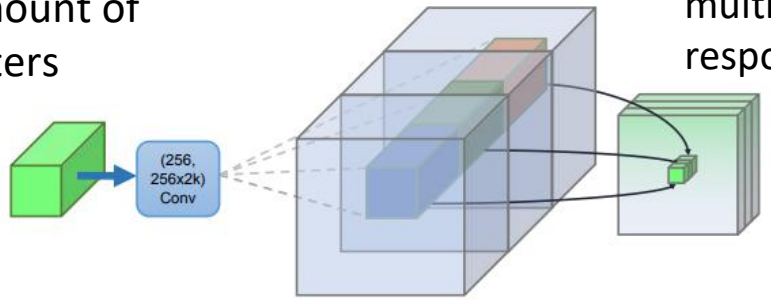


(a) Cross Correlation Layer

SiamRPN

huge amount of parameters

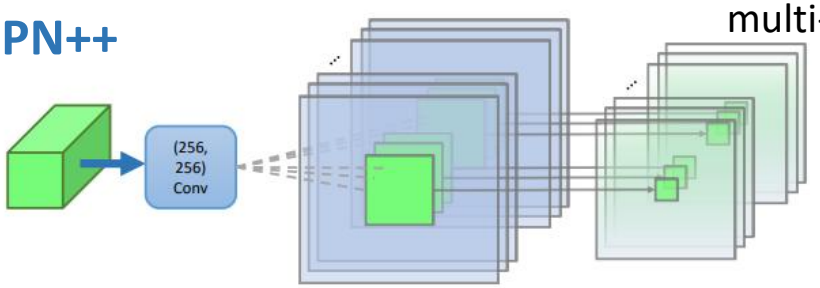
multi-channel response maps



(b) Up-Channel Cross Correlation Layer

SiamRPN++

multi-channel response maps



(c) Depth-wise Cross Correlation Layer

SiamRPN++

- Ablation study

Finetune: the backbone is trained offline

EAO: expected average overlap

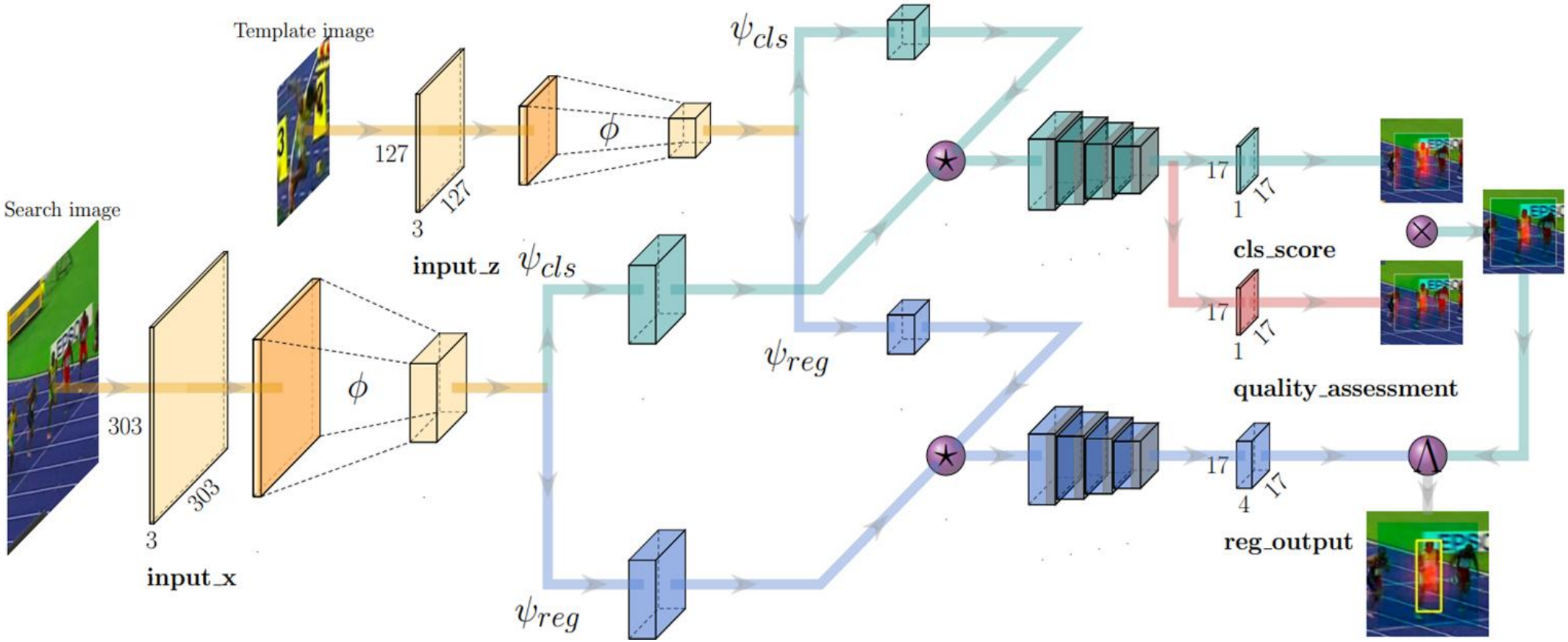
BackBone	L3	L4	L5	Finetune	Corr	EAO VOT2018	EAO OTB2015
AlexNet					UP	0.332	0.658
					DW	0.355	0.666
ResNet-50	✓	✓	✓		UP	0.371	0.664
	✓	✓	✓	✓	UP	0.390	0.684
ResNet-50	✓			✓	DW	0.331	0.669
		✓		✓	DW	0.374	0.678
			✓	✓	DW	0.320	0.646
	✓	✓		✓	DW	0.346	0.677
	✓		✓	✓	DW	0.336	0.674
		✓	✓	✓	DW	0.383	0.683
ResNet-50	✓	✓	✓		DW	0.395	0.673
	✓	✓	✓	✓	DW	**0.414**	**0.696**

SiamFC++: Towards Robust and Accurate Visual Tracking with Target Estimation Guidelines (AAAI2020)

- Goal
  - Decouple two sub-tasks: classification and state estimation
  - Anchor free
  - Quality assessment

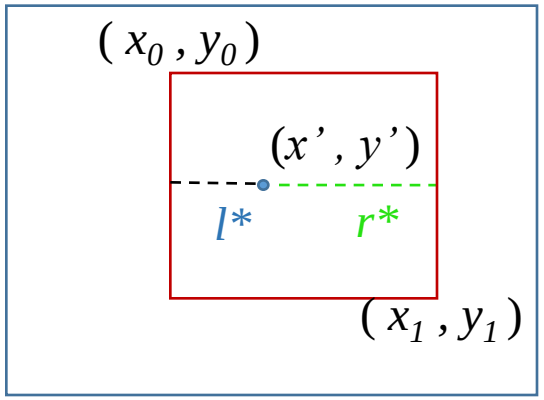
SiamFC++

$$f_i(z,x) = \psi_i\left(\phi\left(z\right)\right) \star \psi_i\left(\phi\left(x\right)\right), i \in \{\text{cls}, \text{reg}\}$$



: feature extractor ■: classification branch ■: regession branch ■: quality assessment
●: operation ⋆: cross-correlation ×: element-wise production Λ: argmax

SiamFC++

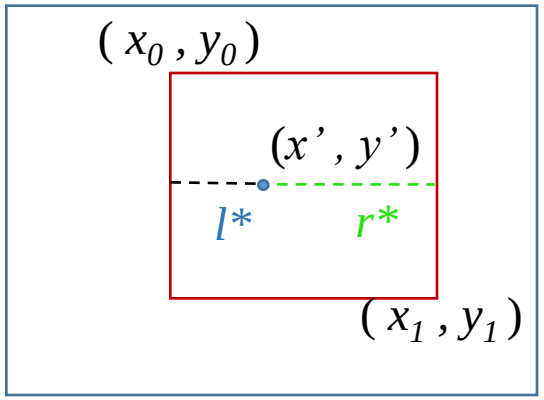


$$f_i(z, x) = \psi_i(\phi(z)) \star \psi_i(\phi(x)) \qquad i \in \{cls, reg\}$$

- $\phi(.)$: for common feature extraction (share parameters)
- $\Psi_i(.)$: task-specific layer $\quad i$: "cls" for classification and "reg" for regression
- $\star$: cross-correlation operation
- **Classification**: (x, y) on class score map is positive if the corresponding location on input image falls into GT bbox
- **Regression**: regression target for (x, y) on the reg map is to predict (l^*, t^*, r^*, b^*) which are the distances from (x', y') to the 4 sides of GT bbox

$$l^* = x' - x_0 \qquad r^* = x_1 - x' \qquad t^* = y' - y_0 \qquad b^* = y_1 - y'$$

SiamFC++



- **Quality assessment**
  - PSS (Prior Spatial Score) $$\mathrm{PSS}^* = \sqrt{\frac{\min(l^*, r^*)}{\max(l^*, r^*)} \times \frac{\min(t^*, b^*)}{\max(t^*, b^*)}}$$

- Loss function

$$L\left(\{p_{x,y}\}, q_{x,y}, \{\boldsymbol{t}_{x,y}\}\right) = \frac{1}{N_{\text{pos}}} \sum_{x,y} L_{\text{cls}}\left(p_{x,y}, c^*_{x,y}\right) + \frac{\lambda}{N_{\text{pos}}} \sum_{x,y} \mathbf{1}_{\{c^*_{x,y}>0\}} L_{\text{quality}}\left(q_{x,y}, q^*_{x,y}\right) + \frac{\lambda}{N_{\text{pos}}} \sum_{x,y} \mathbf{1}_{\{c^*_{x,y}>0\}} L_{\text{reg}}\left(\boldsymbol{t}_{x,y}, \boldsymbol{t}^*_{x,y}\right)$$

- Bbox selection during Inference
  - The score used for final box selection is computed by multiplying the **PSS** with the corresponding predicted **classification score**

Summary

- ROLO
  - LSTM-based tracking
- GOTURN
  - Target regression
- SiamFC
  - Similarity learning
  - Multi-scale test
- SiamRPN
  - Region proposal networks
  - Anchor based
  - Shallow neural networks
- SiamRPN++
  - Region proposal networks
  - Anchor based
  - Deeper networks (Resnet50)
  - Layer-wise aggregation
  - Depth-wise cross-correlation
- SiamFC++
  - Deeper networks (Resnet50)
  - Anchor free
  - Per-pixel prediction
  - End-to-end bounding box regression

Object Tracking

Recurrent Neural Network (RNN)

VOT (Video Object Tracking)

- **No on-line learning:** **ROLO, GOTURN, SiamFC, SiamRPN, SiamRPN++, SiamFC++**

MOT (Multiple Object Tracking)

- **IoU Tracker, SORT, Deep SORT, ByteTrack, FairMOT, Tracktor, Siamese Track-RCNN**

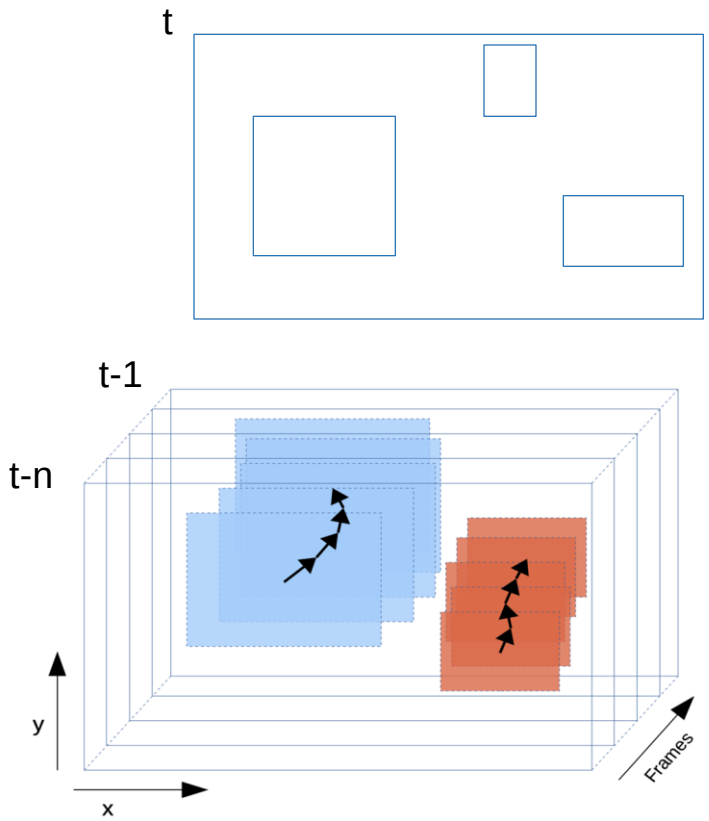
VOS (Video Object Segmentation)

MOT (Multiple Object Tracking)

Tracking-by-detection Paradigm

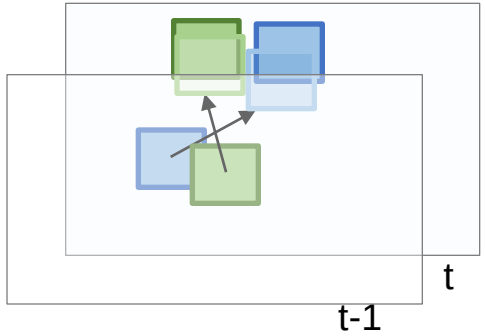
- Object detection network
  - Faster R-CNN, YOLO, RetinaNet, Mask R-CNN, …
- Object tracker (data association)
  - Similarities metrics: motion, appearance, …
  - Commonly used techniques
    - Kalman Filter (motion prediction)
    - Hungarian algorithm (solve the assignment problem)
    - Recurrent Neural Network

IoU Tracker



- Object detector detects objects in the current frame t
- IoU tracker continues a track at frame t-1 by associating it with the detection with the highest IoU and meets threshold σ_{IOU}
  - The tracks without an assigned detection will end.
  - The detections not assigned to an existing track will start a new one.
- Very fast: 100fps
- It works fine based on the assumption
  - The detector has high detection precision and recall
  - Objects of the same identities in consecutive frames have a high overlap IOU which is common when using sufficiently high frame rates.

SORT: Simple Online and Realtime Tracking



- Detector : faster R-CNN(ZFnet), faster R-CNN (VGG)
- Data association
  - Hungarian algorithm (reject if IOU < $IOU_{threshold}$)
  - cost : IOU distance between $\underbrace{\text{detection}}_{\text{at } t}$ and $\underbrace{\text{tracking bbox predicted by Kalman filter}}_{\text{at } t}$
- Performance : good in tracking precision and accuracy, but still with identity switches
- **Kalman filter**
  - State of each target: $x = [\, u , v , s , r , \dot{u} , \dot{v} , \dot{s} \,]$

    (u, v) target center, s: scale, r: aspect ratio u' v' : velocity along u, v directions

    When a detection is associated to a target, the detected bbox (by object detector) and the predicted bbox (by Kalman filter) are weighted to update the target state of Kalman filter; otherwise the state is simply predicted using the linear velocity model.

Hungarian algorithm

Also called **Munkres assignment algorithm**

- Consider there are n agents and n tasks
- Any agent can be assigned to any task, incurring some cost that may vary for different agent-task assignment.
- Goal: to assign exactly one agent to each task and exactly one task to each agent in a way that total cost is minimized.

- Exhausting search for optimal solution: O(N!)
- Hungarian algorithm can reduce time complexity
- #(agent) and #(task) can be different

	Tacklet 1	Tracklet 2	Tracklet 3
Det 1	40	20	15
Det 2	25	30	45
Det 3	60	80	17

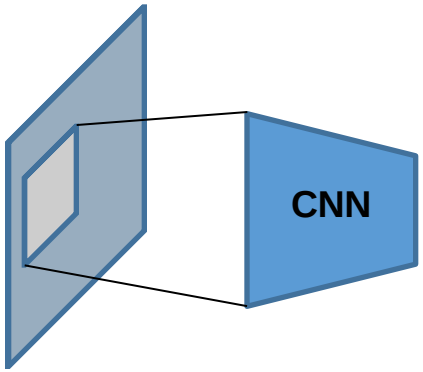
40	20	15
25	30	45
60	80	17

40	20	15
25	30	45
60	80	17

DeepSORT: Simple Online and Realtime Tracking with a Deep Metric

Motion metric (d1)

- Mahalanobis distance between predicted Kalman states (at **t**) and the detected bboxes (at **t**)



Appearance metric (d2)

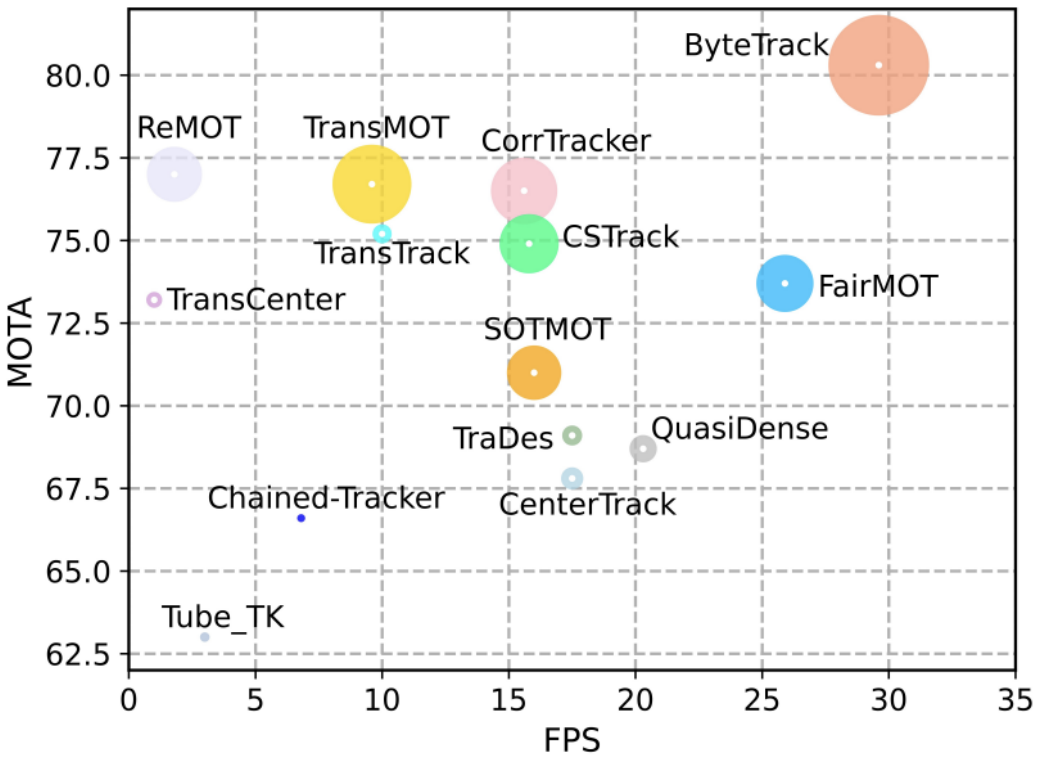
- apply a pre-trained CNN to compute bbox appearance descriptors.
  - for each detected bbox
  - for each track k, keep a gallery of the last L = 100 appearance descriptors
- Cosine distance between each track and detection in appearance
  - Distance = 1 − cos similarity ➔ $1 - r_i^T r_j$ (face embedding *ri* has been L2-normalized)

Data association

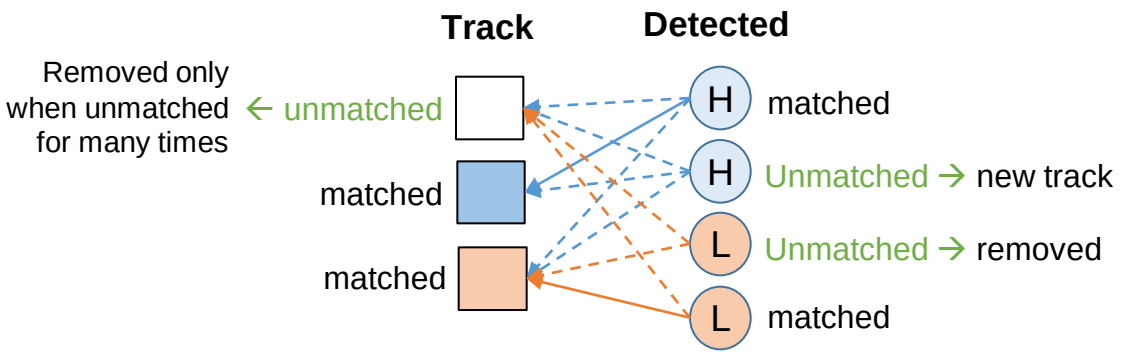
- metric : weighted sum of two metrics $d(i,j) = \lambda d^{(1)}(i,j) + (1-\lambda) d^{(2)}(i,j)$

ByteTrack: Multi-Object Tracking by Associating Every Detection Box

- Most method obtain identities by associating detection boxes whose scores are higher than a threshold.
  - Objects with low scores (e.g., occluded objects) are simply thrown away
- Solution
  - A simple, effective tracking method by associating almost every detection box instead of only high score ones.



ByteTrack



1. Separate detected boxes into high-score and low-score ones
2. Use Kalman filter to predict new locations in current frame for each track
3. Associate the high-score detection boxes to all the tracks.
   - Use Hungarian algorithm, IoU cost + similarity cost
   - unmatched high-score boxes: Initialize new tracks for them
4. Associate the low-score detection boxes to the unmatched tracks
   - Only use IoU cost in the 2nd association because low-score boxes usually contain occlusion or motion blur. Appearance features (similarity cost) are not reliable.
   - Unmatched low-score boxes : deleted and view them as background.
   - Unmatched tracks : put in T_{lost}. They will be removed only after they keep unmatched for more than a certain number of frames (e.g., 30)

FairMOT: On the Fairness of Detection and Re-Identification in MOT

One-shot model

Learns the Detector and Embedding model in a single network. That is, it simultaneously outputs detection results and the embedding of detected boxes

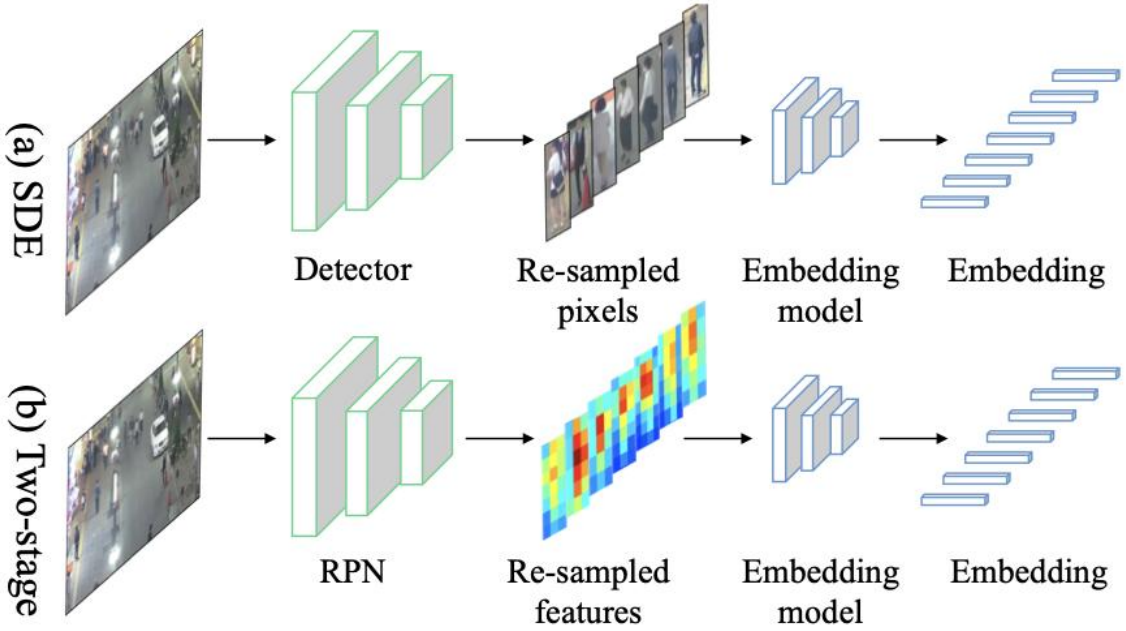
Recall

SDE (Separate Detection and Embedding**)**

The time is the summation of the two tasks, and will increase as the #target increases.

Two-stage model

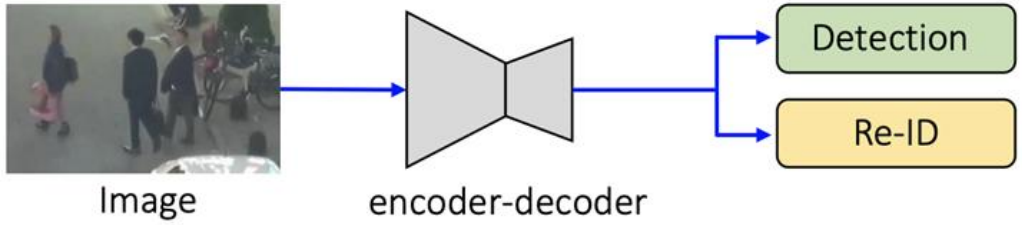
The two tasks share the same set of features, but the time of the 2nd stage still increases as target number increases.

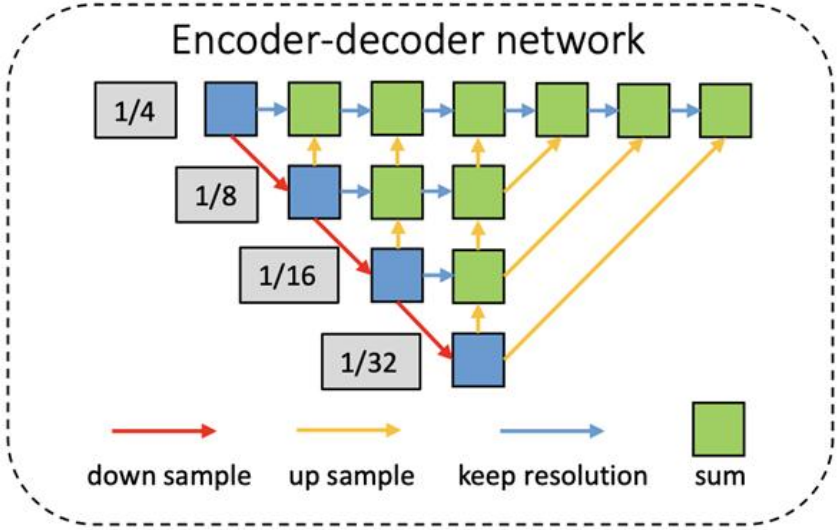


FairMOT

Overview of FairMOT architector

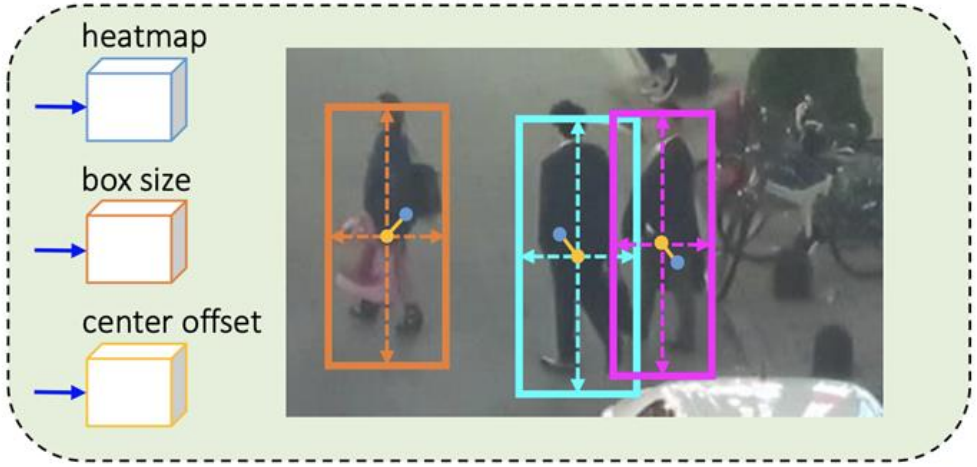
Architecture overview



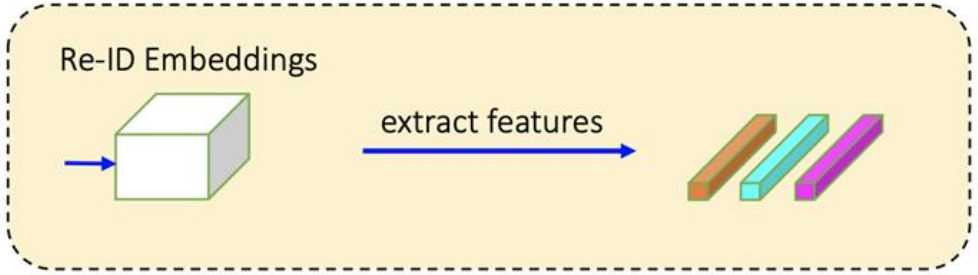


- multi-layer fusion

Detection



Re-ID



FairMOT

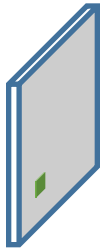
$$M_{xy} = \sum_{i=1}^{N} \exp^{-\frac{(x-\tilde{c}_x^i)^2+(y-\tilde{c}_y^i)^2}{2\sigma_c^2}}$$

$(\tilde{c}_x^i, \tilde{c}_y^i)$: GT object center

N : # of objects

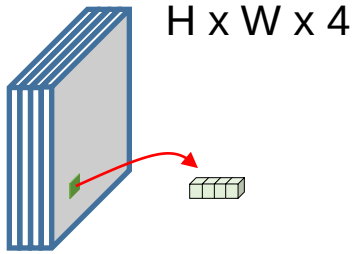
heat map head

- Output dimension: H × W x 1
- GT heatmap: 1 at the GT object center, and then decays exponentially as distance increases
- loss function: focal loss (pixel-wise)



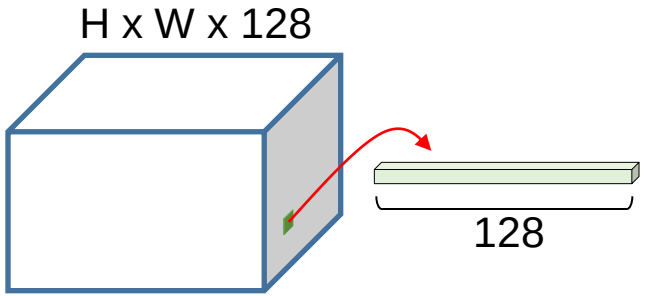
bbox size and center offset head

- Output dimension: H x W x 4
  - bbox size (h, w) and offset (x, y) relative to GT box center
- loss function: L1 (only the cells at object centers are counted)



re-ID embedding branch

- Output dimension: H x W x 128
- Through a classification task (Same identity are same class)
- loss function: cross-entropy



FairMOT

On-line Network Inference

- Use non-maximum suppression (NMS) on heatmap scores to extract the peak keypoints.
- Keep the keypoints with scores are larger than a threshold as estimated object centers.
- At each estimated object center, compute the corresponding bboxes (based on the estimated offsets and box sizes).
- Extract the identity embeddings at the estimated object centers.
- Data association
  - initialize a number of tracklets based on detected boxes in the 1st frame.
  - For subsequent frames, link detected boxes to tracklets by ***bipartite*** matching according to their cosine distances of Re-ID features and bbox IOU.
    - use Kalman Filter to predict tracklet locations in the current frame. If it is too far from the linked detection, set the distance as infinite (to prevent from linking)
  - update the re-ID features of trackers at each time step